

Technical Documentation

Small Graphics Engine

Ivan Džanija

`ivan.dzanija@fer.unizg.hr`

University of Zagreb, Faculty of Electrical Engineering and Computing

June 7, 2026

Version 1.0

Contents

1	Introduction	2
2	Drawing lines on the screen	2
3	Drawing and shading polygons	3
3.1	Component Diagram	5

1 Introduction

This is the technical documentation for the Small Graphics Engine project developed for the laboratory exercise in the Interactive Computer Graphics course at the University of Zagreb. The purpose of this document is to provide an overview of the system architecture, design decisions, and implementation details.

2 Drawing lines on the screen

This section describes the algorithm used for drawing lines on the raster display. The default algorithm for doing such a task is Bresenham's line algorithm. That is also the algorithm implemented here, although only for education purposes, later on the line drawing is done using OpenGL's built-in line drawing capabilities that fully utilize the GPU's capabilities for rasterization. Bresenham's line algorithm here is implemented fully on the CPU, with manual fragment shading. The screen raster is kept in memory as a 2D array of pixels. Here is the code snippet for the manual fragment shading:

```

1 // Raster is kept as
2
3 // Shades a fragment at the given (x, y) coordinates with the specified
  color.
4 int32 Graphics::shade_fragment(int32 x, int32 y, glm::vec3 color) {
5     y = _height - 1 - y; // Invert y-axis
6     if (x >= 0 && x < _width && y >= 0 && y < _height) {
7         _raster(x, y) = color;
8         // _raster[(y * _width * 3) + (x * 3)] = color.x;
9         // _raster[(y * _width * 3) + (x * 3) + 1] = color.y;
10        // _raster[(y * _width * 3) + (x * 3) + 2] = color.z;
11        return 0;
12    }
13    std::cerr << "ERROR: fragment out of bounds: (" << x << ", " << y <<
      ")" << std::endl;
14    return -1;
15 }
```

Listing 1: Manual fragment shading example

While internal representation of the raster is defined as:

```

1 struct Raster {
2     int32 width{};
3     int32 height{};
4     std::vector<glm::vec3> data;
5 };
```

Listing 2: Raster representation example

As for the line drawing algorithm itself more information (such as full derivation) can be found [here](#). The snippet of the line drawing algorithm is as follows:

```

1 // 0 to 90 degrees
2 void bresenham_line_1(eng::Graphics &screen, int32 x0, int32 y0, int32
  x1, int32 y1,
3                     const glm::vec3 &color) {
4     const int32 dy = y1 - y0;
5     const int32 dx = x1 - x0;
6     if (dy <= dx) {
7         const int32 change = dy << 1; // 2 * dy
```

```

8     const int32 correction = -dx << 1;    // -2 * dx
9
10    int32 y_curr = y0;
11    int32 y_cal = -dx;
12    for (int32 x_curr = x0; x_curr <= x1; ++x_curr) {
13        screen.shade_fragment(x_curr, y_curr, color);
14        y_cal += change;
15        if (y_cal >= 0) {
16            ++y_curr;
17            y_cal += correction;
18        }
19    }
20 } else {
21     const int32 change = dx << 1;          // 2 * dx
22     const int32 correction = -dy << 1;    // -2 * dx
23
24     int32 x_curr = x0;
25     int32 x_cal = -dy;
26     for (int32 y_curr = y0; y_curr <= y1; ++y_curr) {
27         screen.shade_fragment(x_curr, y_curr, color);
28         x_cal += change;
29         if (x_cal >= 0) {
30             ++x_curr;
31             x_cal += correction;
32         }
33     }
34 }
35 }

```

Listing 3: Bresenham's line algorithm example

3 Drawing and shading polygons

In this section we describe the algorithm for drawing and shading polygons. To be more precise we are talking about convex polygons, as the algorithm for concave polygons is more complex and is not implemented here. In this section we are still doing the drawing and shading on the CPU, with manual fragment shading as this is this educational implementation and most of the Graphics API's (such as OpenGL) provide built-in functionality for drawing and shading polygons that fully utilize the GPU's capabilities. First thing we are doing is providing a user with a way to define a polygon. When the user presses the left mouse button the application records the position of the mouse cursor and adds it to the list of polygon vertices. If the vertex just added would form a non-convex polygon the user is alerted in the console. The checking is done by seeing if all the vertices of the polygon are on the same side of the line formed by any two vertices of the polygon. If that is not the case then the polygon is not convex. Here is the code snippet for checking if a polygon is convex:

```

1 [[nodiscard]] std::optional<bool> test_convex() const {
2     if (_elements.size() < 3) {
3         return std::nullopt;    // Not enough vertices to determine convexity
4     }
5     uint8 negative = 0;
6     uint8 positive = 0;
7     for (size_t i = 0; i < _elements.size(); ++i) {
8         const auto &curr_edge =
9             _elements[(i + _elements.size() - 1) % _elements.size()].edge;
10        const auto &next_point = _elements[(i + 1) % _elements.size()].
            point;

```

```

11     double dot =
12         (curr_edge.a * next_point.x) + (curr_edge.b * next_point.y) +
13         curr_edge.c;
14     if (dot < 0) {
15         ++negative;
16     }
17     if (dot > 0) {
18         ++positive;
19     }
20     return (negative == 0 || positive == 0);
21 }

```

Listing 4: Convex polygon checking example

The the user finishes defining the polygon by pressing the right mouse button the last line between the last vertex and the first vertex is drawn. After that the polygon is filled using the scanline algorithm. The scanline algorithm works by iterating through each scanline (horizontal line) of the raster and finding the intersection points between the scanline and the left and right edges of the polygon. Specifically for each scanline we are looking for the leftmost right edge and the rightmost left edge. After that we are filling the pixels between those two edges using the Bresenham's line algorithm described in the previous section. The code snippet for the scanline algorithm is as follows:

```

1 void draw_filled(eng::Graphics &screen) {
2     if (_elements.empty()) {
3         // std::cerr << "Polygon has no vertices to draw." << std::endl;
4         return; // No vertices to draw
5     }
6
7     int32 x_min = _elements[0].point.x;
8     int32 x_max = _elements[0].point.x;
9     int32 y_min = _elements[0].point.y;
10    int32 y_max = _elements[0].point.y;
11    double L;
12    double R;
13
14    for (const auto &element : _elements) {
15        x_min = std::min(x_min, element.point.x);
16        x_max = std::max(x_max, element.point.x);
17        y_min = std::min(y_min, element.point.y);
18        y_max = std::max(y_max, element.point.y);
19    }
20
21    for (int32 y = y_min; y <= y_max; ++y) {
22        L = static_cast<double>(x_max);
23        R = static_cast<double>(x_min);
24        for (const auto &curr : _elements) {
25            if (curr.edge.a == 0) {
26                continue; // Skip horizontal edges
27            }
28            double x = (-curr.edge.b * y - curr.edge.c) / static_cast<double>
29                >(curr.edge.a);
30            if (test_orientation() == PolyOrientation::CLOCKWISE) {
31                if (curr.left) {
32                    L = std::min(L, x);
33                } else {
34                    R = std::max(R, x);
35                }
36            }
37        }
38        // Fill the scanline between L and R
39    }
40 }

```

```
34     }
35   } else {
36     if (curr.left) {
37       R = std::max(R, x);
38     } else {
39       L = std::min(L, x);
40     }
41   }
42 }
43 eng::draw_line(screen, static_cast<int32>(L + 0.5F), y,
44                  static_cast<int32>(std::round(R + 0.5F)), y);
45 }
46 // Draw starting vertex in red and outline in blue for clarity
47 screen.shade_fragment(_elements[0].point.x, _elements[0].point.y,
48                      glm::vec3(1, 0, 0));
49 draw_outline(screen);
50 }
```

Listing 5: Scanline algorithm example

3.1 Component Diagram

More details go here...